Here is the complete, step-by-step developer workflow for creating **3 new sections** and previewing their layouts in the development environment.

Step 1: Create the React Block Components

First, create your frontend block components under `src/components/blocks/`.
For example, if you are creating a new block named `ErpFeaturesList.tsx`: Create **`src/components/blocks/ErpFeaturesList.tsx`**:

tsx

```
import React from 'react';
interface ErpFeaturesListProps {
 content: {
   title?: string;
   items?: string[];
 };
}
export function ErpFeaturesList({ content }: ErpFeaturesListProps) {
 return (
   <section className="py-20 bg-white">
     <div className="max-w-6xl mx-auto px-6">
       <h2 className="text-3xl font-black text-[#4B2A63] mb-6">
         {content.title || "Default Features Title"}
       </h2>
       <ul className="space-y-3">
         {content.items?.map((item, i) => (
           <li key={i} className="text-slate-600 font-medium flex
items-center gap-2">
             <span className="w-2 h-2 rounded-full bg-[#FFD54F]" />
             {item}
           </li>
         ))}
       </ul>
     </div>
   </section>
 );
}
```

Step 2: Register the Section Metadata

Next, you need to register your new block metadata in the CMS system so that the Admin Panel and dropdown selectors recognize it.
Open **`src/lib/cms/section-registry.ts`** and add your definitions to the `SECTION_REGISTRY` array:

typescript

```
import { ListPlus } from 'lucide-react'; // Import any Lucide icon
export const SECTION_REGISTRY = [
 // ... existing definitions
 {
   type: 'erp-features-list',
```

```
    label: 'ERP Features List',
    description: 'Dynamic bullet-point features section',
    icon: ListPlus,
    color: 'bg-purple-50 text-[#4B2A63]',
    defaultVariant: 'default',
    supportsVariants: false,
  },
];
```

Step 3: Wire Up the Dynamic Section Renderer

Next, tell the dynamic page compiler how to map the registered type to your React component. Open **`src/components/cms/SectionRenderer.tsx`**:

1. Import your new component:
2. typescript
3. `import { ErpFeaturesList } from '@/components/blocks/ErpFeaturesList';`
4. Add a `case` switch statement:
5. typescript
6. `switch (section.type) {`
7. `  // ... existing cases`
8. `  case 'erp-features-list':`
9. `    return <ErpFeaturesList content={section.content} />;`
10. `}`

Step 4: Preview and Test the Sections in Development

You have **three excellent methods** to test and preview your sections live during the development phase:

Method A: Direct Admin Panel UI (No Coding Required)

Since your sections are fully registered in the registry, they will automatically appear in your Admin Panel!

1. Start your local dev server: `npm run dev`.
2. Open your browser to `/admin/templates`.
3. Select the template you want to modify, and click **Edit** (or add them inside any active Page).
4. Click **Add Section** in the UI—your new **"ERP Features List"** will be available in the dropdown!
5. Add the section, modify its JSON configuration content directly inside the panel, and click **Save Changes**.
6. Back on the list view, click the **Preview** button to launch your live template layout rendering the new blocks instantly inside a realistic mock container.

Method B: Database Seeding (Best for Standard Templates)

If you want these new blocks to automatically populate in your database with default content whenever you reset the database:

1. Update your seed script (e.g. `scripts/seed-erp-template.ts`).
2. Add a new `template_sections` record corresponding to the template:
3. typescript
4. `await db.insert(template_sections).values({`
5. `  templateId: template.id,`
6. `  type: 'erp-features-list',`
7. `  variant: 'default',`
8. `  contentJson: {`
9. `    title: "Operational Excellence Features",`
10. `    items: ["Real-time reporting", "Automated ledgers", "Multi-site syncing"]`
11. `  },`
12. `  orderIndex: 6 // Position order`
13. `});`
14. Run `npm run db:seed-erp-template` in your terminal to instantly refresh the database with your new blocks pre-seeded.

Method C: Quick Sandbox Dev Route

If you want to view a block in isolation before attaching it to a database or template structure:

1. Temporarily add the block inside your dev route (e.g. inside `src/app/page.tsx` or a custom sandbox route like `/sandbox/page.tsx`):
2. tsx
3. `import { ErpFeaturesList } from '@/components/blocks/ErpFeaturesList';`
4. `export default function Sandbox() {`
5. `  return (`
6. `    <ErpFeaturesList`
7. `      content={{`
8. `        title: "Sandbox Test Features",`
9. `        items: ["Module 1", "Module 2", "Module 3"]`
10. `      }}`
11. `    />`
12. `  );`
13. `}`
14. Navigate to `http://localhost:3000/sandbox` in your browser to inspect alignment, typography, margins, and animations in real time.